

1) Sistemas Operativos - Generalidades

a) ¿Quién es el encargado de administrar el tiempo de la CPU y cuál es su función principal dentro del sistema?

El encargado de administrar el tiempo de la CPU es el denominado "Scheduler", el cual forma parte del sistema operativo. Su función principal consiste en decidir qué proceso o hilo se ejecuta en un momento dado y por cuanto tiempo. Esto es así con el objetivo de optimizar el uso del procesador cumpliendo con tareas como maximizar el rendimiento, garantizar equidad entre procesos, cumplir prioridades, entre otras.

b) Explique a que se denomina "Contexto de Ejecución". ¿Cuáles son las variables intervinientes?

En un sistema operativo, el "**Contexto de Ejecución**" se refiere al conjunto de información que describe el estado de un proceso o hilo en un momento dado, permitiendo que el sistema pueda pausar su ejecución y reanudarla posteriormente sin perder información.

Variables intervinientes en el contexto de ejecución en un sistema operativo:

- **Contexto del Proceso:** Incluye toda la información necesaria para suspender y reanudar la ejecución de un proceso de forma transparente.
- **Contexto del Hilo:** En sistemas con Múltiples hilos, cada hilo tiene su propio contexto, incluyendo registros, pila, etc.
- **Contexto del Sistema:** Información global del sistema y del hardware necesario para gestionar la ejecución múltiple.

c) ¿A que se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real?

Un Sistema Operativo de Tiempo Real (RTOS) es un sistema operativo diseñado para gestionar tareas con restricciones de tiempo estrictas y predecibles. A diferencia de los sistemas operativos de propósito general, un RTOS se enfoca en garantizar que las tareas se ejecuten dentro de plazos definidos, lo que lo hace esencial en aplicaciones donde el tiempo es crítico, como sistemas embebidos, controles industriales y dispositivos médicos.

Razones para usar un RTOS:

- **Comportamiento predecible y determinista:**

Los RTOS aseguran que las tareas se ejecuten en el tiempo esperado, lo cual es crucial en sistemas donde la sincronización es fundamental.

- **Gestión eficiente de recursos:**

Permiten una mejor administración de la CPU, memoria y otros recursos del sistema, optimizando el rendimiento general.

- **Manejo de interrupciones:**

Un RTOS responde rápidamente a las interrupciones, minimizando el tiempo de procesamiento y asegurando la ejecución oportuna de tareas críticas.

- **Mayor capacidad de respuesta:**

Los RTOS ofrecen tiempos de respuesta más rápidos y menores latencias en comparación con los sistemas operativos generales, lo que es vital en aplicaciones con requisitos de tiempo real.

- **Desarrollo simplificado:**

Un RTOS ayuda a dividir la aplicación en tareas más pequeñas y manejables, facilitando el desarrollo y la depuración del código.

- **Reutilización de código:**

La modularidad que ofrece un RTOS permite reutilizar código en diferentes proyectos, lo que ahorra tiempo y esfuerzo en el desarrollo.

- **Reducción de riesgos:**

En aplicaciones críticas para la seguridad, el RTOS proporciona una base sólida y confiable, lo que reduce el riesgo de fallos y facilita la certificación.

- **Soporte y mantenimiento:**

Muchos RTOS comerciales ofrecen soporte técnico y actualizaciones, lo que facilita el desarrollo y la solución de problemas.

- **Adaptabilidad a diferentes plataformas:**

Los RTOS pueden adaptarse a una amplia gama de microcontroladores y procesadores, lo que permite a los desarrolladores elegir la opción que mejor se ajuste a sus necesidades.

2) Kernel de FreeRTOS

a) Explique y desarrolle las características de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.

En **FreeRTOS**, una **tarea** (task) es una función que se ejecuta de manera concurrente con otras funciones dentro de un sistema embebido multitarea. Estas tareas tienen su propio contexto de ejecución y son gestionadas por el **planificador** (scheduler) de FreeRTOS, que determina cuál tarea debe ejecutarse en un momento dado, dependiendo de las prioridades y otros parámetros definidos.

```
void task_random(void *params) {  
    while(1) {  
        // Genera un numero aleatorio (0 a 100)  
        uint32_t random_number = rand() % 100;
```

```

// Mando por cola
xQueueSend(queue_random, &random_number, portMAX_DELAY);

// Bloqueo tarea para no saturar
vTaskDelay(pdMS_TO_TICKS(500));
}
}

```

b) Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.

Estados de una Tarea en FreeRTOS

1. Ready (Listo)

Este es el estado en el que una tarea se encuentra cuando está lista para ejecutarse, pero espera su turno según la **prioridad** y la disponibilidad del procesador. En este estado, la tarea espera que el planificador la seleccione para su ejecución.

2. Running (Ejecutándose)

La tarea se encuentra en este estado cuando el planificador la selecciona para ser ejecutada. Esto ocurre cuando el procesador está ejecutando la tarea y no está siendo interrumpido por ninguna otra tarea de mayor prioridad. El estado **Running** es exclusivo, es decir, solo puede haber una tarea ejecutándose en un momento dado.

3. Blocked (Bloqueada)

Una tarea entra en el estado de **bloqueada** cuando está esperando un evento o condición para continuar. Por ejemplo, una tarea puede bloquearse esperando una respuesta de una cola o esperando un semáforo. Una vez que se cumple la condición esperada, la tarea vuelve al estado **Ready**.

4. Suspended (Suspendida)

Una tarea puede ser suspendida manualmente por otro hilo o tarea mediante una función del sistema operativo. Esto significa que la tarea deja de ser gestionada por el planificador y no se ejecutará hasta que sea reanudada explícitamente. Las tareas suspendidas no se ejecutan, ni siquiera si tienen la mayor prioridad.

5. Deleted (Eliminada)

Una tarea puede ser eliminada explícitamente por otra tarea, ya sea cuando ya no es necesaria o cuando se desea liberar los recursos que estaba utilizando. La tarea eliminada ya no existe en el sistema y no puede ser reanudada.

Transiciones de los Estados

A continuación, se describen las transiciones más comunes entre los estados de una tarea y las funciones que permiten estas transiciones:

1. De Ready a Running (Ejecutándose):

- **Transición:** El planificador selecciona la tarea para ejecutarse.
- **Condición:** La tarea es seleccionada porque está lista para ejecutarse y tiene la mayor prioridad disponible.
- **Función:** El planificador de FreeRTOS maneja automáticamente esta transición. No es necesario invocar una función para esto.

2. De Running a Ready (Listo):

- **Transición:** La tarea deja de ejecutarse, pero está lista para ejecutarse nuevamente.
- **Condición:** Puede ser preemptada por una tarea de mayor prioridad, o la tarea ha terminado su tiempo de ejecución.
- **Función:**
 - **vTaskDelay():** Suspende la tarea por un período específico, lo que permite que otras tareas se ejecuten.
 - **taskYIELD():** Fuerza a la tarea actual a ceder el control, permitiendo que el planificador ejecute otra tarea de igual o mayor prioridad.

3. De Running a Blocked (Bloqueada):

- **Transición:** La tarea espera un evento o recurso antes de poder continuar.
- **Condición:** La tarea no puede continuar su ejecución hasta que se cumpla una condición (por ejemplo, esperar en una cola o semáforo).
- **Función:**
 - **xQueueReceive():** La tarea se bloquea esperando un dato de una cola.
 - **xSemaphoreTake():** La tarea se bloquea esperando un semáforo.
 - **vTaskDelay():** Bloquea la tarea durante un período específico.

4. De Blocked a Ready (Listo):

- **Transición:** La tarea estaba bloqueada esperando un recurso y ahora está disponible.
- **Condición:** Un evento que la tarea estaba esperando ha ocurrido (por ejemplo, un semáforo ha sido liberado).
- **Función:**

- **xQueueSendFromISR()** o **xSemaphoreGiveFromISR()**: Desbloquean a la tarea desde una interrupción.
- **xQueueSend()** o **xSemaphoreGive()**: Desbloquean la tarea cuando el recurso se ha liberado.

5. De Running a Suspended (Suspendida):

- **Transición:** La tarea es suspendida manualmente.
- **Condición:** Una tarea puede ser suspendida para evitar que se ejecute hasta que se indique lo contrario.
- **Función:**
 - **vTaskSuspend()**: Suspende la tarea manualmente.

6. De Suspended a Ready (Listo):

- **Transición:** La tarea suspendida es reanudada.
- **Condición:** La tarea es reanudada cuando se decide que puede continuar.
- **Función:**
 - **vTaskResume()**: Reanuda la tarea suspendida.

7. De Running a Deleted (Eliminada):

- **Transición:** La tarea termina su ejecución y se elimina.
- **Condición:** La tarea finaliza o es eliminada explícitamente.
- **Función:**
 - **vTaskDelete()**: Elimina la tarea, liberando los recursos asociados.

8. De Suspended a Deleted (Eliminada):

- **Transición:** Una tarea suspendida es eliminada.
- **Condición:** Se decide eliminar la tarea mientras está suspendida.
- **Función:**
 - **vTaskDelete()**: Puede ser usada para eliminar la tarea en cualquier momento, incluso si está suspendida.

Funciones Importantes para Manejar Estados de Tareas:

- **xTaskCreate()**: Crea una tarea, poniéndola en el estado **Ready**.

- **vTaskDelay()**: Suspende la tarea durante un periodo, poniéndola en el estado **Blocked**.
- **vTaskSuspend()**: Suspende una tarea manualmente, moviéndola al estado **Suspended**.
- **vTaskResume()**: Reanuda una tarea suspendida, llevándola al estado **Ready**.
- **vTaskDelete()**: Elimina una tarea, poniéndola en el estado **Deleted**.

c) ¿A que nos referimos con el concepto de “tareas de procesamiento continuo”? ¿y a que nos referimos con el concepto de “tareas manejadas por eventos”?

Tareas de Procesamiento Continuo

Las tareas de procesamiento continuo son aquellas que se ejecutan de manera indefinida, de forma periódica o continua, sin depender de un evento específico. Estas tareas están diseñadas para realizar operaciones que deben ejecutarse de manera constante, como actualizar un sensor, monitorear una señal, o mantener una comunicación constante con otro dispositivo.

Tareas Manejadas por Eventos

Las tareas manejadas por eventos son aquellas que no se ejecutan de manera continua, sino que esperan a que ocurra un evento o condición específica antes de ejecutarse. El procesamiento solo se activa cuando el evento esperado se produce, como una señal, un mensaje o un temporizador. Esto permite que el sistema sea más eficiente, ya que las tareas solo consumen recursos (como tiempo de CPU) cuando realmente se requiere su ejecución.

d) ¿Cuáles son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explicar.

Retardo por Tiempo (Delay o Timeout)

Una tarea puede bloquearse voluntariamente durante un periodo de tiempo específico. Esto se usa para realizar tareas periódicas, evitar el uso continuo del procesador, o simplemente “dormir” la tarea temporalmente. Por ejemplo: `vTaskDelay(ticks)`

Recepción de Datos desde una Cola (Queue)

Una tarea puede bloquearse esperando a que llegue un dato a una **cola**. Esto es útil para desacoplar el productor (que envía datos) del consumidor (que los procesa). Por ejemplo: `xQueueReceive(cola, &dato, tiempo_espera)`.

Espera por un Semáforo o Mutex

Una tarea puede bloquearse si intenta tomar un semaforo binario, un semaforo contador, o un mutex que está ocupado o no disponible. Se usa para sincronización o control de acceso a recursos compartidos. Por ejemplo: `xSemaphoreTake(xBinarySemaphore, portMAX_DELAY);`

Esperando por un Elemento en un Buffer o Stream Buffer

Las stream buffers y message buffers son mecanismos de comunicación más eficientes para transferencias de datos entre tareas o desde una ISR. Una tarea puede bloquearse si intenta recibir datos y no hay ninguno disponible. Por ejemplo: `xStreamBufferReceive(xBuffer, rxData, sizeof(rxData), portMAX_DELAY);`

e) ¿Cuál es la función de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algún momento? Explicar

La Idle Task es una parte esencial de FreeRTOS. Su función principal es mantener el sistema en funcionamiento continuo cuando no hay tareas activas, liberar recursos, y permitir optimizaciones como modos de bajo consumo. Gracias a esta tarea, el procesador nunca queda sin ejecutar código, lo cual es vital en sistemas embebidos en tiempo real.

Si hay tareas listas (Ready), el planificador elige la de mayor prioridad. Si no hay ninguna tarea lista para ejecutar, el sistema ejecuta la Idle Task, nunca hay un estado donde “no se ejecuta nada”. El sistema siempre tiene al menos la Idle Task como respaldo.

3) Desarrolle como es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.

¿Cómo se almacenan los datos en una Cola?

1. Buffer Interno de Memoria Contigua

- Cuando se crea una cola, el kernel de FreeRTOS reserva una porción de memoria contigua para almacenar los elementos.
- Este buffer interno contiene n elementos de tamaño fijo, donde n es la cantidad de elementos especificada y cada uno tiene un tamaño dado (en bytes).

2. No se almacenan punteros, sino copias de los datos

- FreeRTOS copia los datos al buffer de la cola.
- Esto es importante: los datos no se pasan por referencia, sino por valor.
- Esto evita problemas de concurrencia si el productor y consumidor acceden a la misma variable en memoria.

3. Índices de lectura y escritura (punteros de cola)

FreeRTOS mantiene dos índices:

- Índice de lectura (front): apunta al próximo elemento a recibir.
- Índice de escritura (rear): apunta al siguiente espacio disponible para enviar un dato.

4. Control de concurrencia (Thread Safety)

El acceso a las colas es seguro ante concurrencia porque FreeRTOS usa mecanismos internos para garantizar que dos tareas o una tarea e ISR no corrompan el contenido del buffer.

- Internamente usa bloqueos del scheduler o listas de espera para sincronizar el acceso.
- También gestiona automáticamente si la cola está llena (al enviar) o vacía (al recibir).

5. Gestión de memoria

- Las colas son gestionadas por el kernel. Si se crea dinámicamente con `xQueueCreate()`, la memoria es asignada por el heap (normalmente con `pvPortMalloc()`).
- Si se elimina con `vQueueDelete()`, se libera esa memoria.

El almacenamiento de datos en una cola de FreeRTOS se basa en un buffer de memoria contigua y circular, donde los datos se copian (no se referencian) para asegurar seguridad y aislamiento entre tareas. El kernel maneja internamente los punteros, el control de concurrencia y la sincronización, haciendo que las colas sean una herramienta poderosa y segura para comunicar tareas y manejar eventos en sistemas embebidos en tiempo real.

4) Explique utilizando un ejemplo con código el por qué es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

En FreeRTOS, las colas (queues) se utilizan frecuentemente para la comunicación entre tareas. Un escenario común es tener múltiples tareas escritoras que envían datos a una cola, y una sola tarea lectora que consume esos datos. Este patrón es muy útil cuando tenemos varias fuentes de datos que deben ser procesadas por una tarea única o centralizada.

Este patrón tiene varias ventajas, entre ellas:

1. **Centralización del procesamiento:** Una sola tarea lectora puede realizar todo el procesamiento o análisis de los datos, mientras que las tareas escritoras simplemente colocan los datos en la cola. Esto centraliza la lógica y simplifica la gestión.
2. **Eficiencia:** Las tareas escritoras pueden ser tareas de bajo consumo que simplemente generan o recopilan datos, y la tarea lectora puede ser la encargada de procesarlos cuando sea conveniente.
3. **Sincronización simplificada:** Solo necesitamos gestionar la sincronización de los datos de entrada (tareas escritoras) hacia una cola, y la tarea lectora puede operar sobre estos datos de manera sencilla.

Bloqueo por Escritura

El bloqueo por escritura se refiere a que las tareas escritoras se bloquean si la cola está llena. Es decir, si la cola ha alcanzado su capacidad máxima y no hay espacio disponible para colocar un nuevo elemento, la tarea que intente escribir se bloquea hasta que haya espacio disponible

Bloqueo por Lectura

El bloqueo por lectura se refiere a que la tarea lectora se bloquea si intenta leer de una cola vacía, es decir, cuando no hay datos disponibles para leer. La tarea lectora se bloquea hasta que un productor (tarea escritora) coloque nuevos datos en la cola.

```
#include "FreeRTOS.h"

#include "task.h"

#include "queue.h"

#include <stdio.h>

#define QUEUE_SIZE 5 // Tamaño de la cola

#define NUM_WRITERS 3 // Número de tareas escritoras


// Declaración de la cola global

xQueueHandle xQueue;


// Función que será ejecutada por las tareas escritoras

void vTaskWriter(void *pvParameters)
{
    int writerId = (int) pvParameters;

    int data = 0;

    while(1)
    {
        data = writerId * 10; // Asignamos un valor único por tarea
        printf("Writer %d: Enviando dato %d\n", writerId, data);

        if (xQueueSend(xQueue, &data, portMAX_DELAY) == pdPASS)
        {
```

```

        printf("Writer %d: Dato %d colocado en la cola\n", writerId, data);
    }

    vTaskDelay(pdMS_TO_TICKS(1000)); // Espera antes de enviar otro dato
}

// Función que será ejecutada por la tarea lectora
void vTaskReader(void *pvParameters)
{
    int receivedData;

    while(1)
    {
        if (xQueueReceive(xQueue, &receivedData, portMAX_DELAY) == pdPASS)
        {
            printf("Reader: Recibido dato %d de la cola\n", receivedData);
        }
    }
}

int main(void)
{
    // Crear la cola con tamaño de 5 y elementos de tamaño int
    xQueue = xQueueCreate(Queue_SIZE, sizeof(int));

    if (xQueue == NULL)
    {
        printf("Error al crear la cola\n");
        return 1;
    }

    // Crear las tareas escritoras
    for (int i = 0; i < NUM_WRITERS; i++)

```

```

{
    xTaskCreate(vTaskWriter, "Writer", configMINIMAL_STACK_SIZE, (void *) i, 1, NULL);
}

// Crear la tarea lectora
xTaskCreate(vTaskReader, "Reader", configMINIMAL_STACK_SIZE, NULL, 2, NULL);

// Iniciar el scheduler
vTaskStartScheduler();

return 0;
}

```

5) A la hora de enviar datos compuestos: explique cómo se lleva a cabo dicho proceso y por qué no genera conflictos con la definición de la macro del recurso. ¿Cuál sería la implementación si los datos fueran “grandes”, lo que ocasionaría un problema con la memoria RAM?

¿Cómo se envían datos compuestos a una cola en FreeRTOS?

```

typedef struct {
    int id;
    float temperatura;
    char nombreSensor[20];
} SensorData_t;

```

1. Definir la cola con el tamaño de la estructura

Cuando se crea la cola, FreeRTOS necesita saber el tamaño de cada elemento, ya que los datos se copian en un buffer interno:

```

QueueHandle_t xCola;

xCola = xQueueCreate(10, sizeof(SensorData_t)); // 10 estructuras

```

2. Enviar datos

Al enviar un dato a la cola, se pasa la dirección del dato. FreeRTOS copia el contenido al buffer interno, no el puntero.

```

SensorData_t datoSensor = { .id = 1, .temperatura = 22.5, .nombreSensor = "Temp1" };

xQueueSend(xCola, &datoSensor, portMAX_DELAY);

```

3. Recibir datos

La tarea receptora debe tener una estructura igual para recibir el dato.

```
SensorData_t datoRecibido;
```

```
xQueueReceive(xCola, &datoRecibido, portMAX_DELAY);
```

¿Por qué esto no genera conflictos con la definición del recurso?

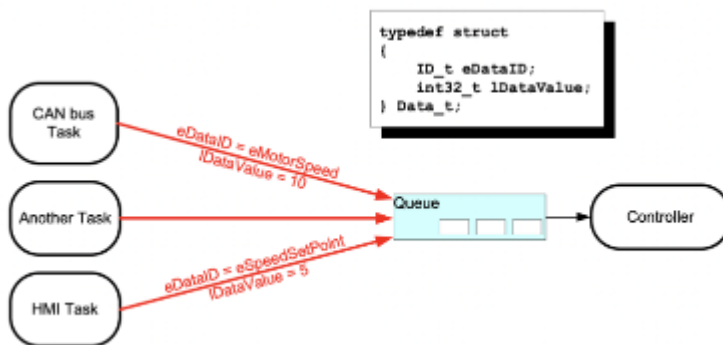
Porque al momento de crear la cola, FreeRTOS reserva memoria interna para almacenar exactamente la cantidad y tamaño de elementos especificados.

Problema: ¿Qué pasa si los datos son "grandes"?

Si los datos compuestos son muy grandes la cola va a consumir mucha RAM porque guarda una copia de cada elemento, recordando que la RAM en microcontroladores suele ser muy limitada.

La solución podría ser enviar punteros en lugar de estructuras completas, reservando dinámicamente el dato a través del puntero

6) Explicar el funcionamiento del esquema de la fig 1:



Explicación del flujo:

- Tareas (Tasks):** Hay tres tareas principales representadas:
 - CAN bus Task:** Esta tarea probablemente está recibiendo información desde un bus CAN (Controller Area Network) y creando un objeto de datos (eData) que contiene el valor de la velocidad del motor (eMotorSpeed).
 - Another Task:** Otra tarea que puede estar procesando datos o generando valores relacionados con el sistema.
 - HMI Task:** Esta tarea se refiere a la interfaz humana (HMI, Human-Machine Interface), que probablemente recibe datos desde la interfaz de usuario, como el control de la velocidad (SpeedSetPoint).
- Estructura de Datos (Struct):** Los datos que se envían a la cola están definidos por una estructura que contiene:

- eDataID: Identificador del tipo de dato.
 - eDataValue: Valor del dato (como la velocidad del motor o el punto de referencia de la velocidad).
3. **Cola (Queue):** Los datos generados por las diferentes tareas se colocan en una cola, que actúa como un buffer para almacenar la información antes de que llegue al Controlador.
 4. **Controlador (Controller):** El controlador consume estos datos de la cola y, según el contexto, realiza las acciones necesarias, como controlar la velocidad del motor o ajustar parámetros.

7) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Semáforos Mutex”.

```
#include "FreeRTOS.h"
```

```
#include "task.h"
```

```
#include "semphr.h"
```

```
#include <stdio.h>
```

```
/* Declaración del handle del Mutex */
```

```
SemaphoreHandle_t xMutex;
```

```
/* Función simulada para imprimir por UART */
```

```
void UART_Send(const char *msg)
```

```
{
```

```
    // Función de envío por UART
```

```
    printf("%s", msg);
```

```
}
```

```
/* Tarea 1 */
```

```
void vTask1(void *pvParameters)
```

```
{
```

```
    for(;;)
```

```
    {
```

```
        /* Intentar tomar el mutex (espera indefinida) */
```

```

    if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
    {
        UART_Send("Tarea 1: Empiezo a usar la UART\n");
        vTaskDelay(pdMS_TO_TICKS(500)); // Simula trabajo
        UART_Send("Tarea 1: Terminé de usar la UART\n\n");

        /* Liberar el mutex */
        xSemaphoreGive(xMutex);
    }
    vTaskDelay(pdMS_TO_TICKS(1000)); // Espera antes de volver a intentar
}

```

```

/* Tarea 2 */
void vTask2(void *pvParameters)
{
    for(;;)
    {
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
        {
            UART_Send("Tarea 2: Empiezo a usar la UART\n");
            vTaskDelay(pdMS_TO_TICKS(300)); // Simula trabajo
            UART_Send("Tarea 2: Terminé de usar la UART\n\n");

            xSemaphoreGive(xMutex);
        }
        vTaskDelay(pdMS_TO_TICKS(800));
    }
}

```

```

int main(void)
{
    /* Inicializar hardware aquí (UART, GPIO, etc.) */

    /* Crear el mutex antes de las tareas */
    xMutex = xSemaphoreCreateMutex();
    if (xMutex == NULL)
    {
        // Error: No hay memoria para crear el mutex

        for(;;);
    }

    /* Crear tareas */
    xTaskCreate(vTask1, "Task 1", configMINIMAL_STACK_SIZE, NULL, 1, NULL);
    xTaskCreate(vTask2, "Task 2", configMINIMAL_STACK_SIZE, NULL, 1, NULL);

    /* Iniciar el scheduler */
    vTaskStartScheduler();

    /* Si llega aquí, hubo un problema */
    for(;;);
}

```

8) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

La UART va a ser gestionada por una tarea “UART Manager” que recibe mensajes a través de una cola.

Las demás tareas no escriben directo a UART, sino que envían cadenas a la cola.

Vamos a tener:

Tarea Baja → prioridad 1 → manda un mensaje largo a la cola y demora.

Tarea Media → prioridad 2 → hace trabajo que no usa la cola.

Tarea Alta → prioridad 3 → intenta mandar un mensaje a la cola cuando la baja todavía está ocupada.

En este esquema, la inversión de prioridad se da si la tarea de baja prioridad está en medio de enviar mensajes y la de alta debe esperar mientras la media sigue ejecutándose.

Si el acceso a la cola está protegido por un mutex, FreeRTOS aplicará la inversión de prioridad

```
#include "FreeRTOS.h"
```

```
#include "task.h"
```

```
#include "queue.h"
```

```
#include "semphr.h"
```

```
#include <stdio.h>
```

```
#define QUEUE_LENGTH 5
```

```
#define QUEUE_ITEM_SIZE sizeof(char *)
```

```
/* Cola para mensajes a la UART */
```

```
QueueHandle_t xQueue;
```

```
/* Mutex para proteger el envío a UART */
```

```
SemaphoreHandle_t xUartMutex;
```

```
/* Simulación de envío UART */
```

```
void UART_Send(const char *msg)
```

```
{
```

```
    printf("%s", msg);
```

```
}
```

```
/* --- Tarea que gestiona la UART (recurso compartido) --- */
```

```
void vUartTask(void *pvParameters)
```

```
{
```

```

char *msg;

for(;;)
{
    if (xQueueReceive(xQueue, &msg, portMAX_DELAY) == pdTRUE)
    {
        /* Proteger el acceso a UART con Mutex */
        if (xSemaphoreTake(xUartMutex, portMAX_DELAY) == pdTRUE)
        {
            UART_Send(msg);
            xSemaphoreGive(xUartMutex);
        }
    }
}

/* --- Tarea Baja Prioridad --- */
void vLowTask(void *pvParameters)
{
    char *msg = "Baja: Mensaje largo, usando UART...\n";
    for(;;)
    {
        xQueueSend(xQueue, &msg, portMAX_DELAY);
        vTaskDelay(pdMS_TO_TICKS(2000)); // Simula trabajo largo
    }
}

/* --- Tarea Media Prioridad --- */
void vMediumTask(void *pvParameters)
{
    for(;;)
    {

```

```

        UART_Send("Media: Trabajando\n");

        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

/* --- Tarea Alta Prioridad --- */
void vHighTask(void *pvParameters)
{
    char *msg = "Alta: Necesito la UART \n";
    for(;;)
    {
        vTaskDelay(pdMS_TO_TICKS(1000)); // Espera para que Baja ocupe el recurso
        xQueueSend(xQueue, &msg, portMAX_DELAY);
    }
}

int main(void)
{
    /* Crear cola */
    xQueue = xQueueCreate(Queue_LENGTH, Queue_ITEM_SIZE);
    if (xQueue == NULL) { for(;;); }

    /* Crear mutex */
    xUartMutex = xSemaphoreCreateMutex();
    if (xUartMutex == NULL) { for(;;); }

    /* Crear tareas */

    xTaskCreate(vUartTask, "UART", configMINIMAL_STACK_SIZE, NULL, 4, NULL); // Gestor de
UART, prioridad alta para vaciar cola

    xTaskCreate(vLowTask, "Baja", configMINIMAL_STACK_SIZE, NULL, 1, NULL);

```

```

xTaskCreate(vMediumTask, "Media", configMINIMAL_STACK_SIZE, NULL, 2, NULL);

xTaskCreate(vHighTask, "Alta", configMINIMAL_STACK_SIZE, NULL, 3, NULL);

/* Iniciar scheduler */
vTaskStartScheduler();

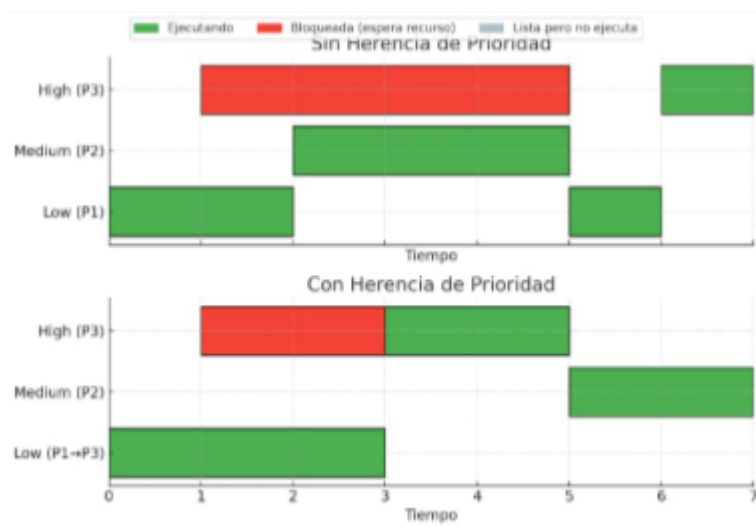
for(;;);
}

```

9) Explicar el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo.

La tarea Low contiene un Mutex que bloquea a High mientras se encuentre en ejecución. Vemos que, si no hubiese herencia de prioridad, la tarea Medium que no necesita del Mutex comenzaría a ejecutarse por tener mayor prioridad que Low. Esto genera que Low no libere el recurso del Mutex, bloqueando la tarea de mayor prioridad (High).

En el diagrama con Herencia de prioridad, la tarea Medium no interrumpe a Low gracias a que la misma incrementa su prioridad para poder liberar el recurso del Mutex, lo que permite que High comience a ejecutarse antes



10) Desarrollar un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador

Concepto de punto muerto

Un punto muerto ocurre cuando dos o más tareas quedan bloqueadas esperando indefinidamente recursos que están retenidos por las otras. Ninguna puede continuar, por lo que el sistema queda “congelado” en esa parte del código.

- Recurso compartido 1 → un mutex que protege el acceso a la UART.
- Recurso compartido 2 → una cola para intercambiar datos.

Tendremos dos tareas:

1. Tarea A:

Toma el mutex (UART)

Intenta escribir algo en la cola (pero la cola está llena porque B tiene que leer).

Queda bloqueada esperando espacio en la cola.

2. Tarea B:

Lee de la cola (pero para imprimir necesita el mutex que tiene A).

Queda bloqueada esperando el mutex.

Resultado:

A espera que B libere espacio en la cola.

B espera que A libere el mutex.

Ninguna puede avanzar → punto muerto.

```
#include "FreeRTOS.h"
```

```
#include "task.h"
```

```
#include "queue.h"
```

```
#include "semphr.h"
```

```
#include <stdio.h>
```

```
#define QUEUE_LENGTH 1
```

```
#define QUEUE_ITEM_SIZE sizeof(int)
```

```
QueueHandle_t xQueue;
```

```

SemaphoreHandle_t xUartMutex;

void UART_Send(const char *msg)
{
    printf("%s", msg);
}

/* --- Tarea A --- */
void vTaskA(void *pvParameters)
{
    int value = 42;
    for(;;)
    {
        /* Paso 1: tomar mutex UART */
        if (xSemaphoreTake(xUartMutex, portMAX_DELAY) == pdTRUE)
        {
            UART_Send("A: Tomé mutex UART\n");

            /* Paso 2: enviar a cola (si está llena, queda bloqueada) */
            UART_Send("A: Intento enviar a cola...\n");
            xQueueSend(xQueue, &value, portMAX_DELAY);

            /* Nunca llega acá si la cola está llena y B no puede leer */
            UART_Send("A: Libero mutex UART\n");
            xSemaphoreGive(xUartMutex);
        }
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

```

```

/* --- Tarea B --- */

void vTaskB(void *pvParameters)
{
    int received;

    for(;;)
    {
        /* Paso 1: recibir de la cola */
        xQueueReceive(xQueue, &received, portMAX_DELAY);

        /* Paso 2: para imprimir necesita el mutex que tiene A */
        UART_Send("B: Intento tomar mutex UART...\n");
        if (xSemaphoreTake(xUartMutex, portMAX_DELAY) == pdTRUE)
        {
            printf("B: Leí %d de la cola\n", received);
            xSemaphoreGive(xUartMutex);
        }
    }
}

int main(void)
{
    /* Crear cola (capacidad 1 para forzar bloqueo rápido) */
    xQueue = xQueueCreate(QUEUE_LENGTH, QUEUE_ITEM_SIZE);
    if (xQueue == NULL) { for(;;); }

    /* Crear mutex */
    xUartMutex = xSemaphoreCreateMutex();
    if (xUartMutex == NULL) { for(;;); }
}

```

```
/* Crear tareas */  
xTaskCreate(vTaskA, "TaskA", configMINIMAL_STACK_SIZE, NULL, 2, NULL);  
xTaskCreate(vTaskB, "TaskB", configMINIMAL_STACK_SIZE, NULL, 2, NULL);  
  
/* Iniciar scheduler */  
vTaskStartScheduler();  
  
for(;;;  
}
```